

Manual de usuario del *C-Product Toolbox* *

Pablo Soto-Quiros[†], Samuel Valverde-Sanchez[‡] y Luis Chavarria-Zamora[✉]

^{†,‡}*Escuela de Matemática, Instituto Tecnológico de Costa Rica, Cartago 30101, Costa Rica*

^{†,✉}*Escuela de Ingeniería en Computadores, Instituto Tecnológico de Costa Rica, Cartago 30101, Costa Rica*

Correos electrónicos: [†]jusoto@tec.ac.cr, [‡]savalverde@itcr.ac.cr y [✉]lachavarria@itcr.ac.cr

ORCID: [†]0000-0003-2903-3116, [‡]0009-0004-3023-8455 y [✉]0000-0002-2510-5680

Resumen

El *C-Product Toolbox* es un paquete computacional diseñado para operar con tensores de tercer orden mediante el *c*-producto reducido, una versión optimizada del *c*-producto que se basa en la transformada discreta de coseno. Desarrollado en MATLAB y PYTHON, este software mejora la eficiencia computacional al reducir el uso de memoria y el tiempo de ejecución, ya que emplea exclusivamente aritmética real y evita los cálculos complejos inherentes al *t*-producto, el cual está basado en la transformada discreta de Fourier. Además, incorpora funcionalidades avanzadas no disponibles en herramientas previas, como las versiones tensoriales de la pseudoinversa y la inversa de Drazin.

Índice

1	Introducción	1
2	Descripción y uso de las funciones del <i>C-Product Toolbox</i>	3
3	Guía rápida para instalar el <i>C-Product Toolbox</i> en Matlab	7
4	Guía rápida para instalar el <i>C-Product Toolbox</i> en Python	9

1. Introducción

1.1. Descripción general

El *C-Product Toolbox* es un paquete computacional diseñado para realizar operaciones sobre tensores de tercer orden utilizando una variante del *c*-producto [1] denominada *c-producto reducido* [2]. Este *toolbox* está disponible tanto para MATLAB como para PYTHON, facilitando su adopción por una amplia comunidad de investigadores y profesionales en computación científica, procesamiento de señales.

El *c*-producto reducido se fundamenta en la transformada discreta del coseno (DCT), lo que permite mejorar la eficiencia computacional de las operaciones tensoriales en comparación con otros enfoques, como el *t*-producto [3], que emplea la transformada discreta de Fourier (DFT) y requiere aritmética compleja. Una explicación detallada del *c-producto reducido* se presenta en la Sección 1.4.

1.2. Ventajas del *C-Product Toolbox*

El *C-Product Toolbox* ofrece funcionalidades avanzadas que no están disponibles en otras herramientas similares, incluyendo la pseudoinversa tensorial, la inversa de Drazin tensorial y la resolución de ecuaciones tensoriales por mínimos cuadrados. Todas las funciones han sido implementadas tanto en MATLAB como en PYTHON utilizando las bibliotecas NUMPY y SCIPY. Al estar implementado en PYTHON, permite su integración en entornos de código abierto sin depender exclusivamente de MATLAB. Este *toolbox* es una

*Version 1.0 - Febrero del 2025

alternativa optimizada al *Tensor-Tensor Product Toolbox 2.0* [4], basado en el *t*-producto, que tiene un mayor consumo de recursos computacionales y carece de actualizaciones recientes.

Gracias a su formulación basada en la DCT, el *C-Product Toolbox* ofrece ventajas clave, tales como un menor consumo de memoria y una reducción en el tiempo de ejecución. Además, los experimentos numéricos reportados en [2] demuestran que este *toolbox* mejora significativamente el rendimiento en términos de eficiencia computacional y uso de memoria. Su aplicabilidad ha sido validada en diversas tareas de procesamiento de señales e imágenes, con resultados destacados en la eliminación de ruido en videos en escala de grises.

1.3. Repositorio de GitHub del *C-Product Toolbox*

El código fuente del *C-Product Toolbox* está disponible en el siguiente repositorio de GitHub:

<https://github.com/jusotoTEC/c-product-toolbox>

Observación 1. *El repositorio de GitHub del C-Product Toolbox incluye todos los ejemplos y experimentos numéricos presentados en [2].*

1.4. Definición formal del *c*-producto reducido

En esta sección, presentamos algunos conceptos y definiciones fundamentales relacionados con el *c*-producto reducido.

Un tensor de tercer orden $\mathcal{A} \in \mathbb{R}^{m \times n \times p}$ es una estructura tridimensional que generaliza la noción de matriz, donde cada elemento está dado por $\mathcal{A}(i, j, k) \in \mathbb{R}$, para todo $i = 1, \dots, m$, $j = 1, \dots, n$ y $k = 1, \dots, p$.

Definición 1 ([1]). Sean $\mathcal{A} \in \mathbb{R}^{m \times n \times p}$ y $\mathcal{B} \in \mathbb{R}^{n \times s \times p}$. Definimos el **producto por caras** como el tensor $\mathcal{C} \in \mathbb{R}^{m \times s \times p}$ dado por:

$$\mathcal{C}^{(i)} = (\mathcal{A} \triangle \mathcal{B})^{(i)} = \mathcal{A}^{(i)} \mathcal{B}^{(i)},$$

donde $\mathcal{A}^{(i)}$ y $\mathcal{B}^{(i)}$ representan las *i*-ésimas caras frontales de los tensores $\mathcal{A}$ y $\mathcal{B}$, respectivamente, para todo $i = 1, \dots, p$.

Definición 2 ([1], [5]). El **producto de modo-3** entre un tensor $\mathcal{A} \in \mathbb{R}^{m \times n \times p}$ y una matriz $M \in \mathbb{R}^{q \times p}$ es el tensor $\mathcal{B} \in \mathbb{R}^{m \times n \times q}$ definido como:

$$\mathcal{B} = \mathcal{A} \times_3 M,$$

donde cada elemento de $\mathcal{B}$ se obtiene mediante la transformación:

$$\mathcal{B}(i, j, :) = \text{vec}^{-1}(M \text{vec}(\mathcal{A}(i, j, :))),$$

para todo $i = 1, \dots, m$ y $j = 1, \dots, n$. Aquí, $\text{vec} : \mathbb{R}^{1 \times 1 \times p} \rightarrow \mathbb{R}^p$ es el operador que convierte un vector tubular en el conjunto $\mathbb{R}^{1 \times 1 \times p}$ en un vector columna en el conjunto $\mathbb{R}^p$, y $\text{vec}^{-1} : \mathbb{R}^p \rightarrow \mathbb{R}^{1 \times 1 \times p}$ es su inverso.

Definición 3 ([1]). La **matriz DCT de tipo II** se define como:

$$C_p(i, j) = \sqrt{\frac{2}{p(1 + \delta(i, 1))}} \cos\left(\frac{\pi(2j - 1)(i - 1)}{2p}\right),$$

para $i, j = 1, 2, \dots, p$, donde δ es la función delta de Kronecker, dada por:

$$\delta(i, j) = \begin{cases} 1, & \text{si } i = j, \\ 0, & \text{si } i \neq j. \end{cases}$$

Con estas herramientas establecidas, presentamos ahora la definición del **c-producto reducido**.

Definición 4 ([2]). El **c-producto reducido** entre los tensores $\mathcal{A} \in \mathbb{R}^{m \times n \times p}$ y $\mathcal{B} \in \mathbb{R}^{n \times s \times p}$ se define como:

$$\mathcal{A} *_c \mathcal{B} = L^{-1}(L(\mathcal{A}) \Delta L(\mathcal{B})),$$

donde L y L^{-1} están definidas por $L(\mathcal{A}) = \mathcal{A} \times_3 C_p \in \mathbb{R}^{m \times n \times p}$ y $L^{-1}(\mathcal{A}) = \mathcal{A} \times_3 C_p^T \in \mathbb{R}^{m \times n \times p}$.

2. Descripción y uso de las funciones del *C-Product Toolbox*

En la Tabla 1 se presenta una lista de las funciones implementadas en el *C-Product Toolbox*, junto con una breve descripción, su sintaxis y la sección en la que se detalla su uso.

Función	Descripción	Sintaxis	Referencia
cprod	c-producto reducido	$\mathbf{C} = \text{cprod}(\mathbf{A}, \mathbf{B})$	Sección 2.1
cinprod	Producto interno tensorial	$\mathbf{c} = \text{cinprod}(\mathbf{A}, \mathbf{B})$	Sección 2.2
ceye	Tensor identidad	$\mathbf{I} = \text{ceye}(n, p)$	Sección 2.3
ctransp	Transpuesta tensorial	$\mathbf{B} = \text{ctransp}(\mathbf{A})$	Sección 2.4
csvd	c-SVD	$[\mathbf{U}, \mathbf{S}, \mathbf{V}] = \text{csvd}(\mathbf{A}, \text{opt})$	Sección 2.5
cqr	c-QR	$[\mathbf{Q}, \mathbf{R}] = \text{cqr}(\mathbf{A}, \text{opt})$	Sección 2.6
csqrtt	Raíz cuadrada tensorial	$\mathbf{X} = \text{csqrtt}(\mathbf{A})$	Sección 2.7
ctubalrank	Rango tubal	$\text{trank} = \text{ctubalrank}(\mathbf{A})$	Sección 2.8
cmultirank	Multi-rango	$\text{mrnk} = \text{cmultirank}(\mathbf{A})$	Sección 2.9
cinv	c-inversa	$\mathbf{X} = \text{cinv}(\mathbf{A})$	Sección 2.10
cpinv	c-pseudoinversa	$\mathbf{X} = \text{cpinv}(\mathbf{A})$	Sección 2.11
cdrazin	c-inversa de Drazin y multi-índice	$[\mathbf{X}, \mathbf{t}] = \text{cdrazin}(\mathbf{A})$	Sección 2.12
cnorm	Norma tensorial	$\mathbf{z} = \text{cnorm}(\mathbf{A}, \text{opt})$	Sección 2.13
clowrank	Aproximación de tensor de bajo rango tubal	$\mathbf{A}_r = \text{clowrank}(\mathbf{A}, r)$	Sección 2.14
csvt	Umbralización de valores singulares tensoriales	$\mathbf{D} = \text{csvt}(\mathbf{A}, \mathbf{t})$	Sección 2.15
clsq	Problema de mínimos cuadrados tensoriales	$\mathbf{X} = \text{tlsq}(\mathbf{A}, \mathbf{B}, \mathbf{C})$	Sección 2.16
test_toolbox	Ejemplos numéricos	$\text{test_toolbox}()$	Sección 2.17

Cuadro 1. Funciones disponibles en el *C-Product Toolbox*.

Observación 2. Para una explicación detallada de los fundamentos matemáticos de cada una de las funciones del *C-Product Toolbox* incluidas en la Tabla 1, se recomienda consultar la Sección 2 de [2].

A continuación, se presenta la documentación de ayuda correspondiente a cada una de las funciones del *C-Product Toolbox*.

2.1. cprod

Descripción: Calcula el c-producto reducido entre dos tensores

Sintaxis de la función: $\mathbf{C} = \text{cprod}(\mathbf{A}, \mathbf{B})$

Entrada: $\mathbf{A}$ = tensor de dimensión $m_1 \times n_1 \times p_1$
 $\mathbf{B}$ = tensor de dimensión $m_2 \times n_2 \times p_2$

Salida: $\mathbf{C}$ = tensor de dimensión $m_1 \times n_2 \times p_1$

2.2. cinprod

Descripción: Calcula el producto interno entre dos tensores
 Sintaxis de la función: `c = cinprod(A,B)`
 Entradas `A` = tensor de dimensión $m \times n \times p$
`B` = tensor de dimensión $n \times m \times p$
 Salida: `c` = producto interno entre los tensores `A` y `B`

2.3. ceye

Descripción: Calcula el tensor identidad bajo el c-producto reducido
 Sintaxis de la función: `I = ceye(n,p)`
 Entradas `n` = entero positivo
`p` = entero positivo
 Salida: `I` = tensor de dimensión $n \times n \times p$

2.4. ctransp

Descripción: Calcula la transpuesta de un tensor bajo el c-producto reducido
 Sintaxis de la función: `B = ctransp(A)`
 Entrada: `A` = tensor de dimensión $m \times n \times p$
 Salida: `B` = tensor de dimensión $n \times m \times p$

2.5. csvd

Descripción: Calcula la descomposición SVD de un tensor bajo el c-producto reducido
 Sintaxis de la función: `[U,S,V] = cpinv(A,opt)`
 Entrada: `A` = tensor de dimensión $m \times n \times p$
`opt` = opciones para varios resultados de `U`, `S`, y `V`
 1) 'full': (predeterminado) produce la descomposición completa SVD del tensor, es decir,
 $A = U * S * V^T$, donde:
`U` = tensor de dimensión $m \times m \times p$
`S` = tensor de dimensión $m \times n \times p$
`V` = tensor de dimensión $n \times n \times p$
 2) 'econ': produce la descomposición de 'tamaño económico'. Sea $s = \min(m,n)$.
 Entonces, $A = U * S * V^T$, donde:
`U` = tensor de dimensión $m \times s \times p$
`S` = tensor de dimensión $s \times s \times p$
`V` = tensor de dimensión $n \times s \times p$
 Salida: `U`, `S`, `V`

2.6. `cqr`

Descripción: Calcula la descomposición QR tensorial bajo el c-producto reducido

Sintaxis de la función: `[Q,R] = cqr(A,opt)`

Entrada: A = tensor de dimensión $m \times n \times p$

`opt` = opciones para diversos resultados de Q y R

1) 'full': (predeterminado) produce la descomposición QR tensorial completo, es decir, $A = Q \cdot R$, donde:

Q = tensor de dimensión $m \times m \times p$

R = tensor de dimensión $m \times n \times p$

2) 'econ': produce la descomposición de 'tamaño económico' cuando $m > n$.

Sea $A = Q \cdot R$, donde:

Q = tensor de dimensión $m \times n \times p$

R = tensor de dimensión $n \times n \times p$

Si $m \leq n$, entonces la descomposición 'tamaño económico' es la misma que la descomposición completa

Salida: Q, R

2.7. `csqrtt`

Descripción: Calcula la raíz cuadrada principal de un tensor bajo el c-producto reducido

Sintaxis de la función: `X = csqrtt(A)`

Entrada: A = tensor de dimensión $m \times m \times p$

Salida: X = tensor de dimensión $m \times m \times p$

2.8. `ctubalrank`

Descripción: Calcula el rango tubal de un tensor utilizando el método c-SVD

Sintaxis de la función: `trank = ctubalrank(A)`

Entradas: A = tensor de dimensión $m \times n \times p$

Salida: `trank` = entero no negativo

2.9. `cmultirank`

Descripción: Calcula el multi-rango de un tensor utilizando el c-producto reducido

Sintaxis de la función: `mrank = cmultirank(A)`

Entradas: A = tensor de dimensión $m \times n \times p$

Salida: `mrank` = vector de dimensión p

2.10. `cinv`

Descripción: Calcula la inversa tensorial
bajo el c-producto reducido

Sintaxis de la función: `X = cinv(A)`

Entradas: `A` = tensor de dimensión $m \times m \times p$

Salida: `X` = tensor de dimensión $m \times m \times p$

2.11. `cpinv`

Descripción: Calcula la pseudoinversa tensorial
bajo el c-producto reducido

Sintaxis de la función: `X = cpinv(A)`

Entradas: `A` = tensor de dimensión $m \times n \times p$

Salida: `X` = tensor de dimensión $n \times m \times p$

2.12. `cdrazin`

Descripción: Calcula la inversa de Drazin de un tensor y
el multi-índice de un tensor bajo el c-producto

Sintaxis de la función: `[X,t] = cdrazin(A)`

Entradas: `A` = tensor de dimensión $m \times m \times p$

Salidas: `X` = tensor de dimensión $m \times m \times p$

`t` = vector de dimensión p

2.13. `cnorm`

Descripción: Calcula la norma tensorial bajo el c-producto

Sintaxis de la función: `z = cnorm(A,opt)`

Entradas: `A` = tensor de dimensión $m \times n \times p$

`opt` = opciones para diferentes tipos de normas
tensoriales:

1) 'fro' : norma de Frobenius

2) 'spec': norma espectral

3) 'nuc' : norma nuclear

Salida: `z` = número no negativo

2.14. `clowrank`

Descripción: Calcula la solución del problema tensorial de
bajo tubal-rango utilizando el c-producto reducido

Sintaxis de la función: `Ar = clowrank(A,r)`

Entradas: `A` = tensor de dimensión $m \times n \times p$

`r` = número entero positivo tal que $r \leq \min(m,n)$

Salida: `Ar` = tensor de dimensión $m \times n \times p$

2.15. csvt

Descripción: Calcula la solución del problema de umbralización de valores singulares tensoriales bajo el c-producto reducido

Sintaxis de la función: `X = csvt(A,t)`

Entradas: `A` = tensor de dimensión $m \times n \times p$

`t` = constante positiva

Salida: `X` = tensor de dimensión $m \times n \times p$

2.16. clsq

Descripción: Calcula la solución del problema de mínimos cuadrados tensoriales bajo el c-producto reducido

Sintaxis de la función: `X = clsq(A,B,C)`

Entradas: `A` = tensor de dimensión $m \times n \times p$

`B` = tensor de dimensión $m \times r \times p$

`C` = tensor de dimensión $s \times n \times p$

Salida: `X` = tensor de dimensión $r \times s \times p$

2.17. test_toolbox

Descripción: Ejemplos numéricos para evaluar el rendimiento y las funcionalidades del C-Product Toolbox

Sintaxis de la función: `test_toolbox()`

3. Guía rápida para instalar el *C-Product Toolbox* en Matlab

3.1. Descargar el archivo `cproduct.zip`

1. Visita el siguiente enlace en tu navegador:

https://github.com/jusotoTEC/c-product-toolbox/tree/main/cproduct_matlab

2. Descarga el archivo `cproduct.zip` en tu computadora.
3. Extrae el contenido del archivo. Esto creará una carpeta llamada `cproduct`.

3.2. Agregar el toolbox al path de Matlab

Para que MATLAB reconozca el *C-Product Toolbox*, siga estos pasos:

1. Abra MATLAB.
2. En la ventana de comandos, ejecute:

```
1 addpath('ruta/al/cproduct');
2 savepath;
```

Reemplace 'ruta/al/cproduct' con la ubicación real de la carpeta `cproduct` en su sistema.

3.3. Verificar la instalación

Para comprobar que las funciones están disponibles, ejecute en la línea de comandos:

```
1 which nombre_funcion
```

Por ejemplo:

```
1 which cprod
```

Si la instalación fue exitosa, MATLAB mostrará la ubicación del archivo correspondiente.

3.4. Uso del *C-Product Toolbox* en Matlab

Ahora es posible invocar las funciones del *C-Product Toolbox* de la misma manera que cualquier otra función de MATLAB. La Figura 1 muestra ejemplos de uso de todas las funciones disponibles en el *C-Product Toolbox* dentro del entorno de MATLAB.

```
1 % Ejemplo de aplicacion de las funciones del C-Product Toolbox en MATLAB
2
3 A = rand(10,5,3);           % Tensor de tercer orden generado ...
4                               % ... aleatoriamente con distribucion uniforme
5 B = rand(5,5,3);           % Tensor de tercer orden generado ...
6                               % ... aleatoriamente con distribucion un
7 C = cprod(A,B);             % c-producto entre A y B
8 c = cprod(A,B);             % producto interno tensorial entre A y B
9 I = ceye(10,5);             % Tensor identidad de tamano 10 x 10 x 5
10 At = ctransp(A);           % Transpuesta tensorial de A
11 [U,S,V] = csvd(A);          % c-SVD de A
12 [Q,R] = cqr(A);             % c-QR de A
13 X1 = cqr(A);                % Raiz cuadrada tensorial de A
14 trank = ctubalrank(A);      % Rango tubal de A
15 mrank = cmultitank(A);      % Multi-rango de A
16 X2 = cinv(B);               % c-inversa de B
17 X3 = cpinv(A);              % c-pseudoinversa de A
18 [X4,t] = cdrazin(A);        % c-inversa de drazin de A y multi-indice
19 z1 = cnorm(A,'fro');         % Norma tensorial de Frobenius de A
20 z2 = cnorm(A,'spec');        % Norma tensorial espectral de A
21 z3 = cnorm(A,'nuc');         % Norma tensorial nuclear de A
22 Ar = clowrank(A,3);         % Tensor solucion del problema de ...
23                               % ... aproximacion de tensor con rango-tubal 3
24 X5 = csvt(A,0.5);           % Tensor solucion del problema de ...
25                               % ... umbralizacion de valores singulares ...
26                               % ... tensoriales con tao=0.5
27 A1 = rand(5,4,3);           % Tensor de tercer orden generado ...
28                               % ... aleatoriamente con distribucion uniforme
29 B1 = rand(5,2,3);           % Tensor de tercer orden generado ...
30                               % ... aleatoriamente con distribucion uniforme
31 C1 = rand(2,4,3);           % Tensor de tercer orden generado ...
32                               % ... aleatoriamente con distribucion uniforme
33 X6 = clsq(A1,B1,C1);        % Tensor solucion del problema de minimos ...
34                               % ... cuadrados tensoriales
```

Figura 1. Ejemplo numérico del *C-Product Toolbox* en MATLAB

3.5. (Opcional) Añadir el *C-Product Toolbox* al inicio de Matlab

Si desea que el *toolbox* se cargue automáticamente cada vez que inicie MATLAB, agregue la línea `addpath('ruta/al/cproduct');` al archivo `startup.m`. Para hacerlo:

1. En MATLAB, ejecute:

```
1 edit startup
```

2. Agregue la línea:

```
1 addpath('ruta/al/cproduct');
```

3. Guarde y cierre el archivo.

Con estos pasos, el *C-Product Toolbox* estará listo para usarse en MATLAB.

4. Guía rápida para instalar el *C-Product Toolbox* en Python

Este manual explica cómo descargar e instalar el paquete `cproduct` en PYTHON desde GitHub.

4.1. Descargar el archivo `cproduct.zip`

1. Visita el siguiente enlace en tu navegador:

https://github.com/jusotoTEC/c-product-toolbox/tree/main/cproduct_python

2. Descarga el archivo `cproduct.zip` en tu computadora.
3. Extrae el contenido del archivo. Esto creará una carpeta llamada `cproduct`.

4.2. Instalación del paquete

1. Abre una terminal o símbolo del sistema.
2. Navega hasta la carpeta `cproduct` descomprimida:

```
1 cd ruta/donde/extraiste/cproduct
```

3. Ejecuta el siguiente comando para instalar el paquete en modo editable:

```
1 pip install .
```

4.3. Verificar la instalación

Después de la instalación, abre PYTHON y ejecuta:

```
1 import cproduct as cp
2 print(cp.__name__) # Deberia imprimir "cproduct"
```

4.4. Usar el *C-Product Toolbox* en Python

Para utilizar las funciones de *C-Product Toolbox*, importa el módulo en tu script:

```
1 import cproduct as cp
```

4.5. Uso del *C-Product Toolbox* en Python

Para utilizar las funciones del *C-Product Toolbox*, importa el módulo en tu script:

```
1 import cproduct as cp
```

En la Figura 2 se encuentran el uso de todas las funciones del *C-Product Toolbox* en PYTHON.

```
1 # Ejemplo de aplicacion de las funciones del C-Product Toolbox en Python
2
3 import cproduct as cp          # Importar el C-Product Toolbox
4 import numpy as np            # Importar el Numpy
5
6 A = np.random.rand(10,5,3)    # Tensor de tercer orden generado ...
7                                # ... aleatoriamente con distribucion uniforme
8 B = np.random.rand(5,5,3)     # Tensor de tercer orden generado ...
9                                # ... aleatoriamente con distribucion un
10 C = cp.cprod(A,B);            # c-producto entre A y B
11 c = cp.cprod(A,B)             # producto interno tensorial entre A y B
12 I = cp.ceye(10,5)             # Tensor identidad de tamaño 10 x 10 x 5
13 At = cp.ctransp(A)            # Transpuesta tensorial de A
14 [U,S,V] = cp.csvd(A)          # c-SVD de A
15 [Q,R] = cp.cqr(A)             # c-QR de A
16 X1 = cp.cqr(A)                # Raiz cuadrada tensorial de A
17 trank = cp.ctubalrank(A)       # Rango tubal de A
18 mrank = cp.cmultitank(A)       # Multi-rango de A
19 X2 = cp.cinv(B)               # c-inversa de B
20 X3 = cp.cpinv(A)              # c-pseudoinversa de A
21 [X4,t] = cp.cdrazin(A)         # c-inversa de drazin de A y multi-indice
22 z1 = cp.cnorm(A,'fro')         # Norma tensorial de Frobenius de A
23 z2 = cp.cnorm(A,'spec')        # Norma tensorial espectral de A
24 z3 = cp.cnorm(A,'nuc')         # Norma tensorial nuclear de A
25 Ar = cp.clowrank(A,3)         # Tensor solucion del problema de ...
26                                # ... aproximacion de tensor con rango-tubal 3
27 X5 = cp.csvt(A,0.5)           # Tensor solucion del problema de ...
28                                # ... umbralizacion de valores singulares ...
29                                # ... tensoriales con tao=0.5
30 A1 = np.random.rand(5,4,3)    # Tensor de tercer orden generado ...
31                                # ... aleatoriamente con distribucion uniforme
32 B1 = np.random.rand(5,2,3)    # Tensor de tercer orden generado ...
33                                # ... aleatoriamente con distribucion uniforme
34 C1 = np.random.rand(2,4,3)    # Tensor de tercer orden generado ...
35                                # ... aleatoriamente con distribucion uniforme
36 X6 = cp.clsq(A1,B1,C1)        # Tensor solucion del problema de minimos ...
37                                # ... cuadrados tensoriales
```

Figura 2. Ejemplo numérico del *C-Product Toolbox* en PYTHON

4.6. Desinstalar el paquete

Si deseas eliminar el paquete, usa:

```
1 pip uninstall cproduct
```

■ Referencias

- [1] E. Kernfeld, M. Kilmer y S. Aeron, «Tensor–tensor products with invertible linear transforms», *Linear Algebra and its Applications*, vol. 485, págs. 545-570, 2015.
- [2] P. Soto-Quiros, S. Valverde-Sanchez y L. Chavarria-Zamora, «*C-Product Toolbox*: A computational package for third-order tensor operations based on the reduced *c*-product», *Computational and Mathematical Methods*, 2025, Under review.
- [3] M. E. Kilmer y C. D. Martin, «Factorization strategies for third-order tensors», *Linear Algebra and its Applications*, vol. 435, n.º 3, págs. 641-658, 2011.
- [4] C. Lu, *Tensor-Tensor Product Toolbox*, <https://github.com/canyilu/tproduct>, Carnegie Mellon University, jun. de 2018.
- [5] H. Jin, S. Xu, Y. Wang y X. Liu, «The Moore–Penrose inverse of tensors via the M-product», *Computational and Applied Mathematics*, vol. 42, n.º 6, pág. 294, 2023.